Title, goal statement, and conventions

Title: GradPath

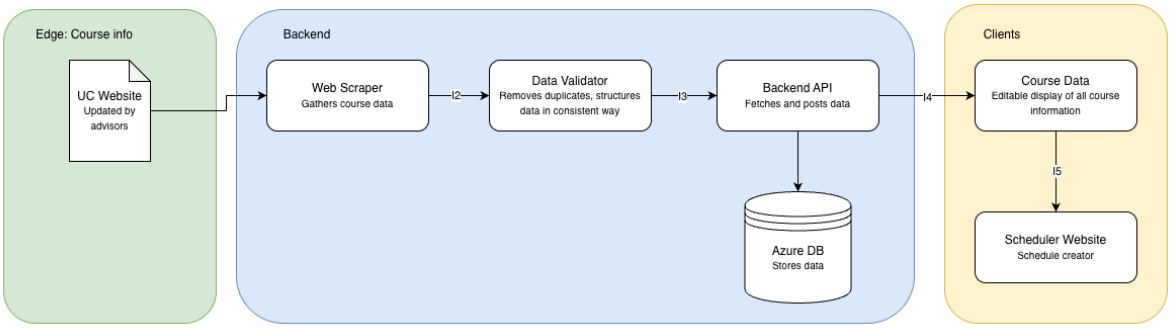
Goal Statement: A scheduler for UC students that uses a degree audit and UC's course catalog to automatically create the fastest possible graduation plan. It considers course prerequisites and semester availability, while allowing students to add, remove, or edit classes and see plans for additional majors or minors.

Conventions:

Block diagram (the D0 diagram)

GradPath

A scheduler for UC students that uses a degree audit and UC's course catalog to automatically create the fastest possible graduation plan. It considers course prerequisites and semester availability, while allowing students to add, remove, or edit classes and see plans for additional majors or minors.



Component responsibility table

Component	Responsibility	Interfaces From/to	Primary Owner
UC Website	Provides official course information updated by advisors	To: Web scraper	
Web Scraper	Extracts course data from UC website and sends it for validation.	From: Web scraper → To: Data Validator	
Data Validator	Cleans/structures course data into a consistent format.	From: Data validator → To: Backend API	
Backend API	Handles data requests between frontend and database.	From: Backend API → To: Azure Database (store/write) , Course Data (display)	
Azure Database	Stores validated course data and student plans.	From: Azure Database (retrieve) → To: Backend API	
Course Data	Retrieves stored course data for the scheduler website, is an editable display of all course information	From: Course Data → To: Scheduler Website	
Scheduler Website	Schedule creator displaying final plan to review	From: Course Data	

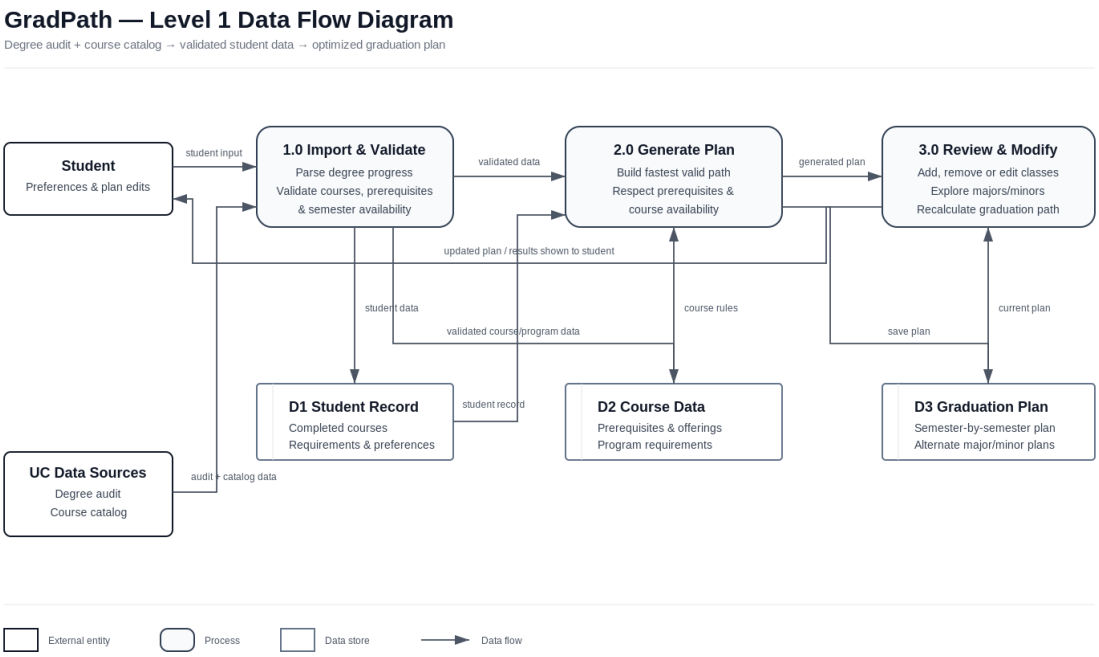
Interface specification table

ID	Connection	Inputs	Outputs	Format	Protocol	On error	Handled by
I1	UC Website -> Web Scraper	Catalog page request	Course catalog page	HTML	HTTPS GET	Retry, then skip page and log	Web Scraper
I2	Web Scraper -> Data Validator	Scraped course data	Validated course data	JSON	Function call	Reject bad record and log	Data Validator
I3	Data Validator -> Backend API	Validated course data	Import status	JSON	HTTPS POST	Return error and validator logs	Backend API
I4	Backend API -> Course Data	Course lookup or edit request	Course data	JSON	HTTPS REST	Show error message	Course Data
I5	Course Data -> Scheduler Website	Selected courses	Semester schedule	JS object	React	Re-fetch courses via I4	Scheduler Website
I6	Backend API -> Azure DB	Course data query	Course data	Table rows	Azure SQL connection	Return 503 to client	Backend API

Payload Example

{ "courses": [{ "courseCode": "CS 3003", "title": "Programming Languages", "credits": 3, "prerequisites": ["CS 2028C"], "termsOffered": ["Fall", "Spring"] }] }

Data-flow diagram



Architecture pattern and justification

Pattern: GradPath uses a mix of three patterns: client-server, pipeline, and layered.

Where each pattern applies:

Pipeline (course data): UC Website -> Web Scraper -> Data Validator -> Backend API -> Azure Database. Course data is scraped, cleaned, then stored in fixed steps.

Client-server (backend): The Backend API serves course data from the Azure Database to the scheduler website in each student's browser.

Layered (app): The scheduler website is split into display (React components), scheduling logic, and data access through the Course Data component.

Scoring patterns against the criteria (1 = poor fit, 3 = strong fit):

Criterion	Layered	Client-server	Pipeline	Micro-services	Embedded
Fit to problem shape	2	3	3	1	1
Team skillset	3	3	2	1	1
Performance and timing	2	2	3	2	2
Scalability	2	3	2	3	1
Hardware constraints	3	3	3	2	1
Total (max 15)	**12**	**14**	**13**	**9**	**6**

Client-server and pipeline score highest, for different parts of the system, so the design uses both. Layered is used inside the app to keep the UI, logic, and data separate.

Justification:

Fit to problem shape - Course data moves one way through clean-up steps, which is a pipeline. Many students reading the same central course data is client-server.

Team skillset - The team is most comfortable with JavaScript, React, and relational databases (ADR-002, ADR-003).

Performance and timing - Scraping and validation run ahead of time, not while a student waits. Students only read already cleaned data, so building a schedule is fast.

Scalability - No login or saved user data (ADR-004), so the server does not track sessions. Azure can raise the database tier if more students use it.

Hardware constraints - Runs on the cloud and in a web browser. No special hardware is needed.

Rejected patterns:

Microservices - Scored 9. Too complex for our team size and semester timeline. Our components are small and do not need separate deployments.

Embedded sensor-actuator - Scored 6. GradPath has no sensors, devices, or real-time loop, so this pattern does not fit.

Decision log

ADR-001

Field	Entry
Title	ADR-001: Simplify API call flow between Backend and course data / Scheduler website
Status	Accepted, Week 6.
Context	The backend originally had two API calls; one to course data and one directly to scheduler website. Caused redundancy and syncing issues.
Options considered	1. Keep both API calls 2. 2. Use a single API call to course data
Decision	Option 2. Simplify one call, from course data to the backend API to retrieve validated course information. The scheduler website then feeds from course data instead of making a second call to the API.
Consequences	Simplifies architecture, reduces redundancy, and ensures consistent data.
Owner and Reviewers	Team GradPath

ADR-002

Field	Entry
Title	ADR-002: Choosing React for the scheduler website
Status	Accepted, Week 6.
Context	The team needed to choose a front-end framework for building the scheduler website. It must display course information dynamically, support interactive scheduling features, and remain easy to maintain throughout the semester. The team also needed a solution that fits within the projects hosting and budget constraints.
Options considered	1. Use plain HTML/CSS/JavaScript without a framework 2. Use React as the main frontend framework 3. Use a server-rendered approach (Flask)
Decision	2. The team chose React as the frontend framework for the scheduler website. React supports the needs of our system. Integrates well with the team's existing skills. Fits within the project's budget; do not require additional paid services.
Consequences	Cleaner component structure for displaying course data. Manageable learning curve for teammates unfamiliar with React. Easier future updates and maintenance.

Owner and Reviewers	Team GradPath

ADR-003

Field	Entry
Title	ADR-003: Choose SQL Database for Data Storage
Status	Accepted, Week 6.
Context	The team needed a reliable, budget friendly way to store validated course and curriculum data
Options considered	1.PostgreSQL 2. MongoDB 3. Azure SQL Database
Decision	Option 3. Azure database. The team is comfortable with relational schemas.
Consequences	Simplifies development; this option fits the budget constraints and team skill set.
Owner and Reviewers	Team GradPath

ADR-004

Field	Entry
Title	ADR-004: Non persistent user data or Login System
Status	Accepted, Week 6.
Context	The team needed to decide whether the scheduler website would store user data across sessions or require login. Adding accounts would increase complexity and introduce security requirements.
Options considered	1. Implement user accounts and persistent storage 2. Use session-only interaction with no saved data
Decision	Option 2. The system will not store user information or require login. Each visit is a temporary session, and all data resets when the page is refreshed.
Consequences	Simple architecture and no authentication needed. Users cannot save schedules between sessions Less of security and privacy risk.
Owner and Reviewers	Team GradPath